



CEN111

OVERVIEW OF C

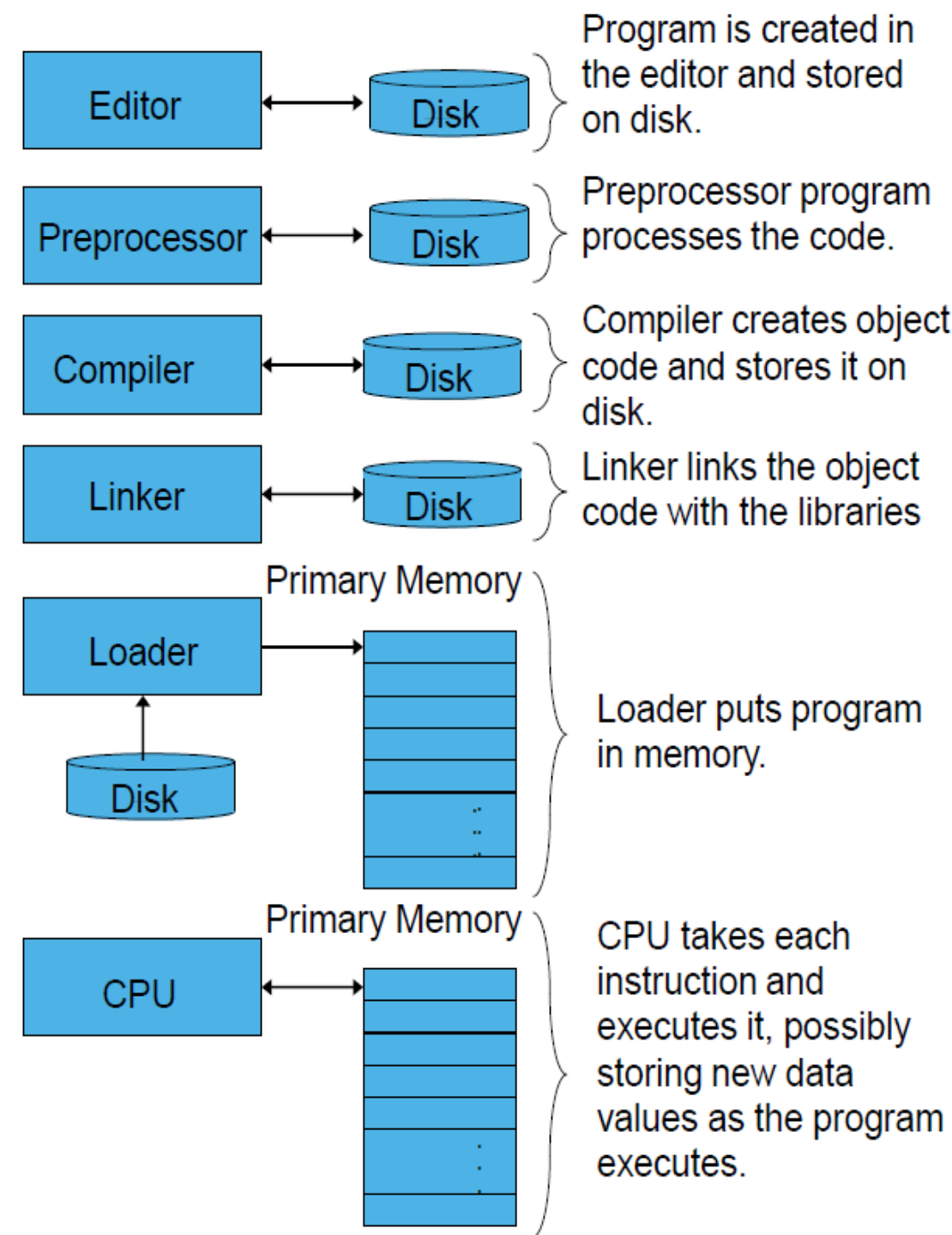


History of C

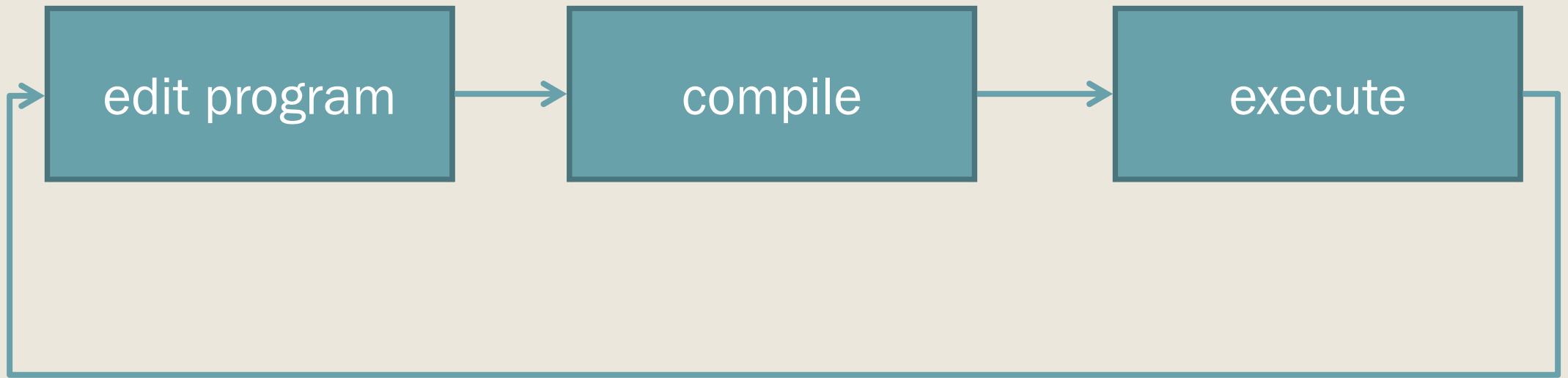
- C is a high-level programming language developed by Denis Ritchie at AT&T Bell Labs in 1972
- Used to write modern operating systems
- Combines some important qualities of high and low level languages
- Allows programmers to develop application software as well as system software

Phases of C Programs

1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute



The Programming Process



- The cycle ends once the programmer is satisfied with the performance of the program

A Simple C Program

```
/* Hello World Example */  
#include <stdio.h>  
int main (void)  
{  
    printf("Hello world!\n");  
    return 0;  
}
```

Comments

```
/* Hello World Example */
```

- `/*` and `*/` indicates that is a comment
- Text surrounded by `/*` and `*/` is ignored by compiler
- Used to describe program
- Comment Line: `//` compiler ignores everything from there until the end of the line

Preprocessor Directives

```
#include <stdio.h>
```

- The preprocessor directives are commands that give instructions to the C preprocessor, whose job it is to modify the text of a C program before its compiled.
- A preprocessor directive begins with a number symbol (#) as its first nonblank character.

Preprocessor Directives

```
#include <stdio.h>
```

- The `#include` directive gives a program access to a certain library
- `#include <stdio.h>` notifies the preprocessor that some names used in the program are found in the standard header file `<stdio.h>`.

#include Directive

- SYNTAX:

```
#include <standard header file>
```

- EXAMPLES:

```
#include <stdio.h>
```

```
#include <math.h>
```

Function main

```
int main (void)
```

- C programs contain one or more functions, exactly one of which must be **main**
- `int` means that `main` "returns" an integer value
- `void` indicates that the function takes no arguments
- Braces, `{` and `}` indicate a block. The bodies of all functions must be contained in braces

main Function Definition

■ SYNTAX:

```
int main(void)
{
    function body
}
```

The printf Function

```
printf("Hello world!\n");
```

- The printf function displays the value of its *format string*
- Entire line called a statement. All statements must end with a semicolon (;)
- Escape character (\) Indicates that **printf** should do something out of the ordinary
- \n is the newline character

The return Statement

```
return 0;
```

- Last line in the main function.
- Transfers control from your program to the operating system.
- The value 0 ,in this case, indicates that your program executed without an error.
- Right brace } indicates end of main has been reached

Language Elements

- **Token:** the smallest element of a program that is meaningful to the compiler.
- Kinds of tokens in C:
 - *Reserved words*
 - *Identifiers*
 - *Constants*
 - *Operators*
 - *Punctuators*

Reserved Words

- a word that has a special meaning in C
- identifiers from standard library and names for memory cells
- appear in lowercase
- cannot be used for other purposes

Reserved Words

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Identifiers

- Identifiers are used for:
 - *Variable names*
 - *Function names*
 - *Macro names*

Identifiers

- An identifier must consist only of letters, digits, and underscores.
- An identifier cannot begin with a digit.
- A C reserved word cannot be used as an identifier.
- An identifier defined in a C standard library should not be redefined.
- Case-sensitive e.g. Ali and ali are two different identifiers.

Identifiers

■ Valid Identifiers

letter_1, letter_2, inches, CENT_PER_INCH,
Hello, variable

■ Invalid Identifiers

1Letter double int TWO*FOUR joe's

Variable Declarations

- Variable
 - *a name associated with a memory cell whose value can change*
- Variable declarations
 - *statements that communicate to the compiler the names of variables in the program and the kind of information stored in each variable*
 - *Variables must be declared before use, a syntax (compile-time) error if these are violated*
- Every variable has a name, a type, a size and a value

Basic Datatypes in C

- Integer int
- Character char
- Floating Point float
- Double precision double
- floating point
- Datatype modifiers
 - *signed / unsigned (for int and char)*
 - *short / long*

Basic Datatypes in C

■ signed char (8 bits)	-127 to +127
■ unsigned char	0 to 255
■ short int (16 bits)	-32,767 to +32,767
■ unsigned short int	0 to 65,535
■ int (32 bits)	-2,147,483,647 to +2,147,483,647
■ unsigned int	0 to 4,294,967,295
■ long int (32-64 bits)	-2,147,483,647 to +2,147,483,647
■ unsigned long int	0 to 4,294,967,295
■ float	$\sim 10^{-37}$ to $\sim 10^{38}$
■ double	$\sim 10^{-307}$ to $\sim 10^{308}$
■ long double	$\sim 10^{-4931}$ to $\sim 10^{4932}$

Variable Declarations

A declaration consists of a data type name followed by a list of (one or more) variables of that type:

- *char c;*
- *int num1, num2;*
- *float rate;*
- *double average;*

Variable Declarations

A variable may be initialized in its declaration.

- *char c = 'a';*
- *int a = 220, b = 448;*
- *float x = 0.00123*
- *double y = 123.00*

Variable Declarations

- Variables that are not initialized may have garbage values.
- Whenever a new value is placed into a variable, it replaces the previous value
- Reading variables from memory does not change them

Simple Macros

- C provides a **#define** directive to define symbolic names for constants:

```
#include<stdio.h>
#define PI 3.14
int main(void)
{
    float area;
    float radius=3;
    area=PI*radius*radius;
    return 0;
}
```

Operators

- Arithmetic operators
- Assignment operator
- Logical operators

Arithmetic Operators

Arithmetic Operator	Meaning	Example
+	addition	5 + 2 is 7 5.0 + 2.0 is 7.0
-	subtraction	5 - 2 is 3 5.0 - 2.0 is 3.0
*	multiplication	5 * 2 is 10 5.0 * 2.0 is 10.0
/	division	5.0 / 2.0 is 2.5 5 / 2 is 2
%	remainder	5 % 2 is 1

Arithmetic Operators

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the <i>innermost</i> pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they’re evaluated left to right.
* / %	Multiplication Division Remainder	Evaluated second. If there are several, they’re evaluated left to right.
+ -	Addition Subtraction	Evaluated third. If there are several, they’re evaluated left to right.
=	Assignment	Evaluated last.

Step 1. $y = 2 * 5 * 5 + 3 * 5 + 7;$ (Leftmost multiplication)

$2 * 5$ is 10



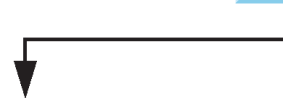
Step 2. $y = 10 * 5 + 3 * 5 + 7;$ (Leftmost multiplication)

$10 * 5$ is 50



Step 3. $y = 50 + 3 * 5 + 7;$ (Multiplication before addition)

$3 * 5$ is 15



Step 4. $y = 50 + 15 + 7;$ (Leftmost addition)

$50 + 15$ is 65



Step 5. $y = 65 + 7;$ (Last addition)

$65 + 7$ is 72



Step 6. $y = 72$ (Last operation—place 72 in y)

Assignment Operator

- variable = expression;
- expressions:
 - *operations*
 - *variables*
 - *constants*
 - *function calls*
- Precedence of the assignment operator is lower than the arithmetic operators

$x = 5 * y + (y - 1) * 44 ;$

expression

statement

Assignment Operator

- an instruction that stores a value of a computational result in a variable
- $x=y$; l-value vs. r-value
- $x+1 = 3$; invalid l-value expression
- l-value usages must refer to a fixed position in the memory

Increment and Decrement Operators

- Postfix Increment/Decrement

int a=3;

a++; \ a=a+1

- The value of the postfix increment expression is determined before the variable is increased

x=a++;

1. *x=a;*

2. *a=a+1;*

Increment and Decrement Operators

- Prefix Increment/Decrement

int a=3;

++a; \ \ a=a+1

- The effect takes place before the expression that contains the operand is evaluated

x=++a;

1. *a=a+1;*

2. *x=a;*

Increment and Decrement Operators

```
/** increment and decrement expressions */  
#include <stdio.h>  
  
int main(void)  
{  
    int a = 0 , b = 0, c = 0;  
    a = ++b + ++c;  
    a = b++ + c++;  
    a = ++b + c++;  
    a = b-- + --c;  
    return 0;  
}
```

Compound Assignment Operator

- `sum=sum+x;`
- can be abbreviated
 - *`sum+=x;`*
- `operand1=operand1 operator operand2`
- `operand1 operator=operand2`

Integer/Float Conversions

- An arithmetic operation between an integer and an integer always yields an integer result .
- An arithmetic operation between a float and a float always yields a float result.
- In an arithmetic operation between an integer and a float, the integer is first promoted to float and then the operation is carried out. Hence, it always yields a float result.
- On assigning a float to an integer, the float is demoted to an integer
- On assigning an integer to a float, it is promoted to a float.

Input and Output

- input operation
 - *an instruction that copies data from an input device into memory*
- output operation
 - *an instruction that displays information stored in memory*
- input/output function
 - *a C function that performs an input or output operation*
- function call
 - *calling or activating function*

The printf Function

- The printf Function displays a line of program output.
- function argument
 - *enclosed in parentheses following the function name*
 - *provides information needed by the function*
- format string
 - *in a call to printf, a string of characters enclosed in quotes, which specifies the form of the output line*
- `printf("<format string> ", <list of variables>)`

The printf Function

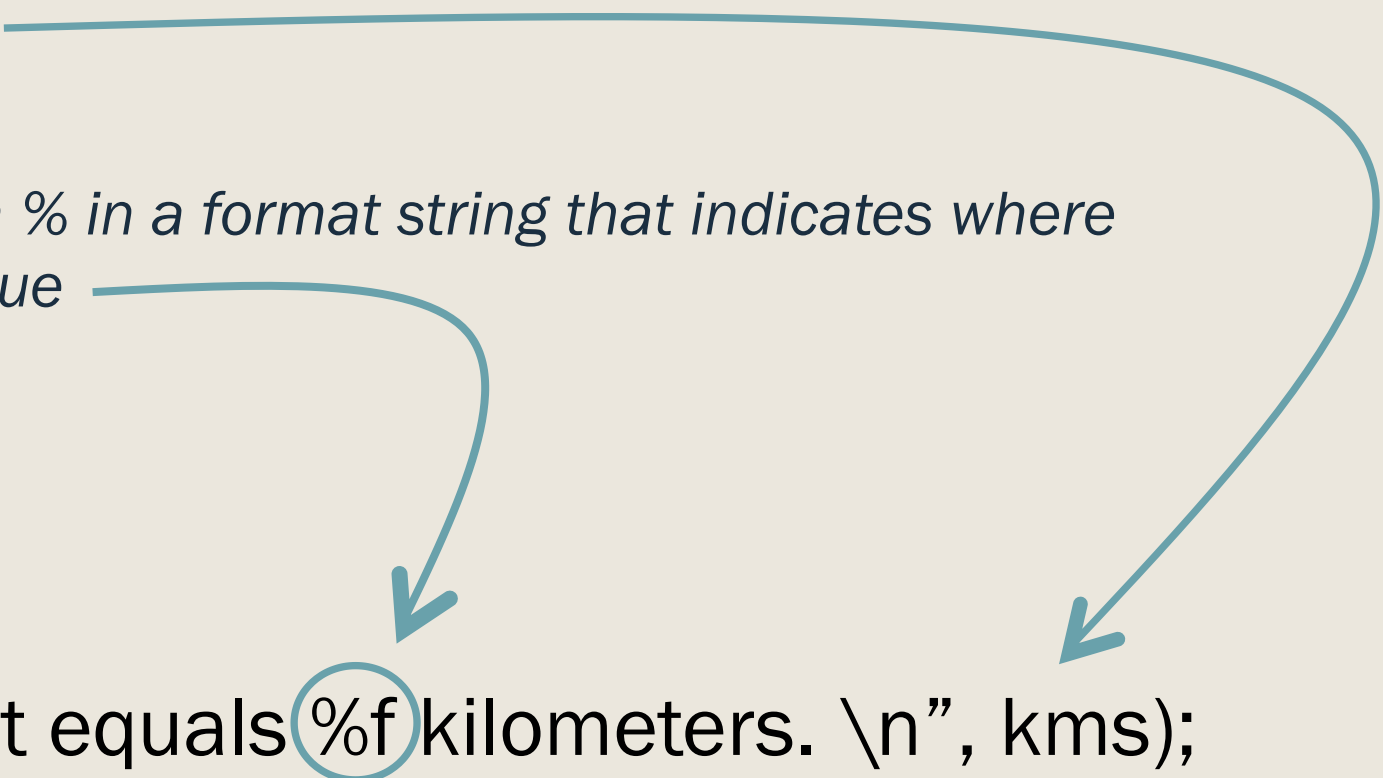
- print list

- *in a call to printf, the variables or expressions whose values are displayed*

- placeholder

- *a symbol beginning with % in a format string that indicates where to display the output value*

printf("That equals%fkilometers. \n", kms);



The printf Function

- Placeholders in format string
 - *%c: the argument is taken to be a single character*
 - *%d: the argument is taken to be an integer*
 - *%f: the argument is taken to be a floating point (float or double)*
 - *%s: the argument is taken to be a string*

The scanf Function

- Copies data from the standard input device (usually the keyboard) into a variable.
- `scanf("<format string>", <address of variables>)`

```
scanf("%d", &number);
```

Input and Output

```
#include <stdio.h>
int main()
{
    int a,b;
    scanf("%d %d",&a,&b);
    printf("%d",a+b);
    return 0;
}
```

Exercise

- Write a C program to convert entered fahrenheit degree to celsius degree.

Homework

- Write a C program to reverse a given 5 digits number.